FIAP

GLOBAL SOLUTION 2026.2

QUEM FAZ

Applied Computer Vision - Detecção Inteligente de EPIs

Professores: Arnaldo Viana e Paulo Rodrigues

Enzo Vazquez Sartorelli - RM 94618
Kaue Pastori Teixeira - RM 98501
Nicolas Nogueira Boni - RM 551965
Guilherme Brazioli - RM 98237
Pedro Oliveira Valotto - RM 551445

11 de novembro de 2026

Resumo

Este relatório apresenta o MVP Quem Faz - Applied Computer Vision, desenvolvido para demonstrar detecção visual de equipamentos de proteção individual em imagens de contexto de construção. O problema abordado é a dificuldade de observar, de forma escalável, indícios de uso de capacete, colete e óculos, sem transformar a solução em certificação de segurança. Foi realizado fine-tuning local de YOLO11n pré-treinado com um subconjunto de três classes do Construction-PPE. As classes helmet, vest e goggles foram remapeadas para capacete, colete e oculos. O conjunto original possui 1.000 imagens; a preparação usou 800 imagens de treino e dividiu deterministicamente as 200 imagens de validação originais em validação e teste, com 100 imagens cada e seed 42. O treinamento válido executou cinco épocas em Windows, Python 3.12, PyTorch 2.7.1+cu118, Ultralytics 8.4.210 e GPU NVIDIA RTX 3060, com imagens de 416 pixels, batch 2, AMP e workers zero. No holdout de 100 imagens e 390 instâncias, nunca empregado no treinamento, foram obtidos precisão 0,8892, recall 0,7177, mAP@50 0,7691 e mAP@50-95 0,4991. A classe colete teve o melhor mAP@50-95, 0,6141; capacete atingiu 0,4762 e oculos 0,4068. O MVP Streamlit aceita imagens, desenha caixas, mostra confiança e oferece processamento local de vídeo por script. As evidências incluem peso treinado, métricas JSON, curvas, matriz de confusão, exemplo anotado e testes automatizados. A solução exige validação adicional em câmeras e cenários reais, sobretudo por oclusão, iluminação, distância e pelo treinamento curto; portanto, deve apoiar, e não substituir, inspeção humana e procedimentos de segurança.

Palavras-chave

visão computacional; detecção de objetos; EPI; YOLO; segurança do trabalho; MVP.

1. Objetivo

Desenvolver e validar um MVP de visão computacional que detecte capacete, colete e óculos de proteção em imagens, usando treinamento customizado de YOLO e inferência local. O objetivo é demonstrar apoio rastreável à inspeção visual de EPI, com métricas reais, interface utilizável e limites explícitos de responsabilidade.

2. Problema e solução

A observação de EPIs em imagens pode apoiar triagem visual, mas não comprova que um equipamento está corretamente vestido, adequado à atividade ou em boas condições. O MVP recebe uma imagem, executa detecção local e retorna caixas delimitadoras, classe e confiança. A decisão de segurança permanece humana.

3. Arquitetura

A interface Streamlit chama o módulo de inferência. Este carrega o peso YOLO11n validado, processa a imagem em 416 pixels e devolve a imagem anotada e uma lista estruturada de detecções. Scripts separados preparam os dados, treinam, validam e processam vídeo localmente.

4. Dados e preparação

Foi usado o Construction-PPE da Ultralytics, licenciado em CC BY 4.0. O dataset original não é redistribuído neste repositório. O script de preparação manteve apenas as classes helmet, vest e goggles e as remapeou para capacete, colete e oculos. Foram geradas 800 imagens para treino, 100 para validação e 100 para teste, com seed 42. O teste foi separado antes do treinamento.

5. Treinamento customizado

Parâmetro	Valor
Modelo	YOLO11n pré-treinado
Épocas	5
Resolução	416 px
Batch	2
Seed	42
Ambiente	Windows, Python 3.12, PyTorch 2.7.1+cu118, RTX 3060
Classes	capacete, colete, oculos

A primeira tentativa de preparação de labels foi inválida por gravar o texto literal \n; ela resultou em zero labels e foi descartada. Após correção, o experimento yolo11n_ppe3_fixed foi reexecutado e é a única origem dos resultados apresentados.

6. Resultados no teste holdout

Métrica	Resultado
Precisão	0.8892
Recall	0.7177
mAP@50	0.7691
mAP@50-95	0.4991

Por classe, o mAP@50-95 foi 0,4762 para capacete, 0,6141 para colete e 0,4068 para oculos. A validação registrou cerca de 9,1 ms de inferência por imagem já carregada. A inferência de exemplo, incluindo carregamento inicial do modelo, retornou um colete com confiança 0,2896 em 3.661,6 ms.

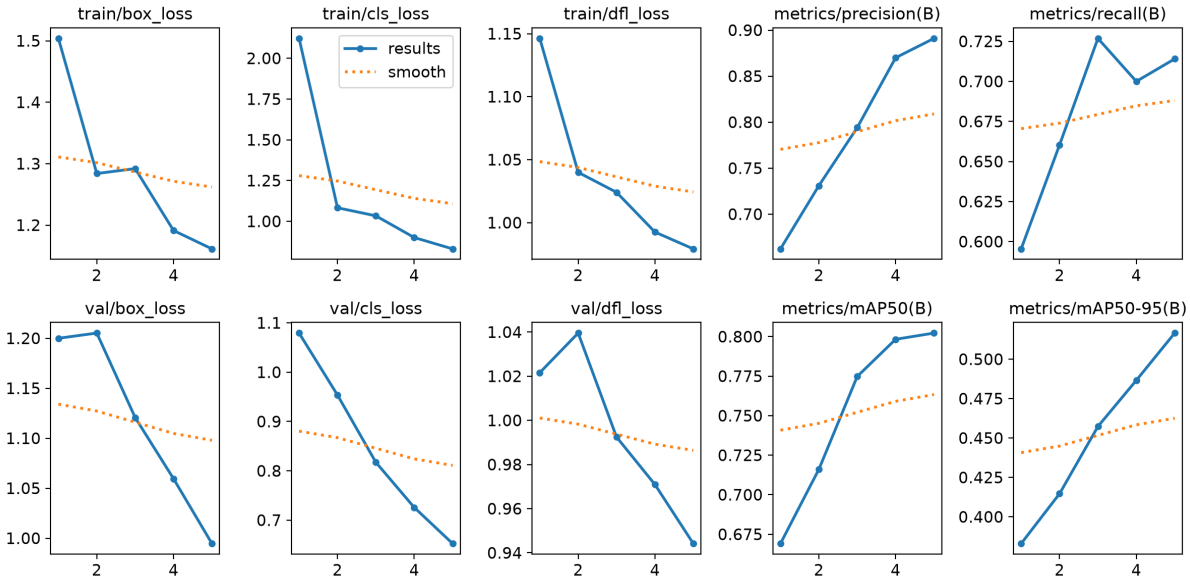


Figura 1 - Curvas do treinamento válido de cinco épocas.

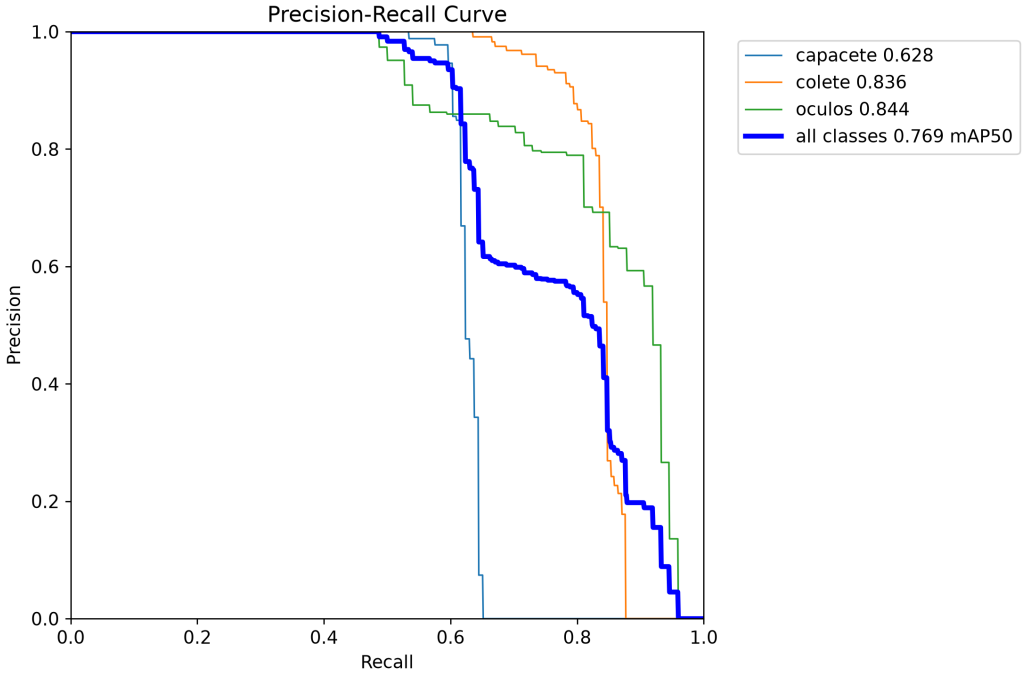


Figura 2 - Curva Precision-Recall no teste holdout.

7. Evidência de inferência



Figura 3 - Inferência real sobre imagem do holdout; a baixa confiança exibida é preservada como evidência, não selecionada como demonstração ideal.

8. Limitações e uso responsável

O conjunto é limitado, o treinamento é curto e o desempenho varia por classe. Objetos pequenos, oclusão, iluminação, distância da câmera e diferenças entre o contexto do dataset e uma obra real podem reduzir a detecção. O sistema não reconhece uso correto, ajuste, estado de conservação ou todas as exigências de segurança. A saída deve ser revisada por pessoas qualificadas e não deve acionar decisão autônoma de segurança.

9. Reprodutibilidade

O repositório contém scripts de preparação, treinamento, validação, interface, testes, peso validado, métricas JSON, gráficos e exemplo anotado. Imagens e labels não são versionados; a obtenção do dataset e sua licença estão documentadas em dataset/README.md. Para verificar o código: python -m unittest discover -s tests -v e python -m compileall -q src training tests.

10. Referências

ULTRALYTICS. Construction-PPE Dataset. Documentação oficial, 2026. Disponível em: https://docs.ultralytics.com/datasets/detect/construction-ppe/. Acesso em: 11 nov. 2026.

ULTRALYTICS. Train Mode e Val Mode. Documentação oficial, 2026. Disponível em: https://docs.ultralytics.com/modes/train/ e https://docs.ultralytics.com/modes/val/. Acesso em: 11 nov. 2026.

PYTORCH. Get Started Locally. Documentação oficial, 2026. Disponível em: https://pytorch.org/get-started/locally/. Acesso em: 11 nov. 2026.

11. Conclusão

Curso: Engenharia de Software. O MVP atingiu o objetivo de demonstrar detecção local de capacete, colete e óculos em imagens, com treinamento customizado, métricas reproduzíveis e interface verificável. As ferramentas empregadas foram Python 3.12, PyTorch, Ultralytics YOLO11n, OpenCV, Streamlit, ReportLab e CUDA. O desempenho do holdout sustenta a demonstração acadêmica, mas a solução requer validação adicional antes de qualquer uso operacional e permanece um apoio à inspeção humana.